Rule-Based Modeling of Cell Signaling

Module 2: Molecular structure and combinatorial complexity

Jim Faeder

MSCBIO 2025 · Introduction to Bioinformatics Programming in Python

Fall 2026

Resources

- **BioNetGen** software, documentation, and tutorials: bionetgen.org
- **VS Code extension**: search the Extensions panel for *BioNetGen*
- **BNG Playground**: write and simulate models in your browser, no installation ruleworld.github.io/bngplayground
- **bngsim**: fast ODE and stochastic simulation from Python bngsim.readthedocs.io · `pip install bngsim bionetgen`
- **Course repository**: models, notebooks, slides and handouts github.com/jrfaeder/RBM_Intro_BioNetGen



Course repository



BNG Playground

The book

Rule-Based Modeling of Biological Systems: Theory and practice with BioNetGen is an open book that develops the material of this course in more depth, with every model, figure, and exercise.

ruleworld.github.io/bionetgen-book

For this module, read:

- Chapter 2: Molecules, patterns, and rules
- Chapter 4: A field guide to rules
- Chapter 6: Network-free simulation



The book

Goals for this module

By the end of this module you should be able to

1. explain why multi-site proteins lead to **combinatorial complexity**
2. represent molecules with **components**, **states**, and **bonds** in BNGL
3. read and write **patterns**, and use them to define observables
4. write **reaction rules** and predict which reactions a rule generates
5. describe how BioNetGen **generates a network** from rules, and when network-free simulation is needed instead

Part 1

From species to molecules

Where Module 1 left off

Our cytokine–STAT–SOCS model had **12 species**, each named by hand:

L	R	LR	S
LRS	LRS_p	S_p	G
GS_p	mSOCS	SOCS	LR·SOCS

To get total pSTAT we had to list every species that contains it:

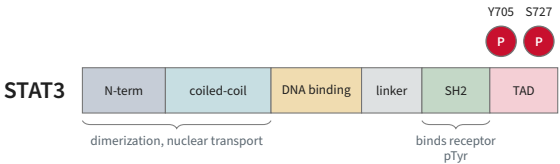
```
Molecules pSTAT LRSp Sp GSp
```

Assumptions we made

- Ligand only leaves an empty receptor
- SOCS and STAT compete for one receptor site
- The receptor is a single chain
- STAT acts as a monomer with one phosphosite

Each of these is false for real cytokine receptors. Relaxing any of them requires new species, and new copies of every reaction they take part in.

Signaling proteins have many interaction sites



n independent two-state sites

$\Rightarrow$ 2^n possible states of one molecule

and complexes multiply the counts of their parts

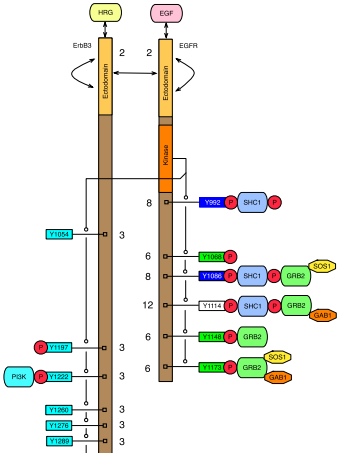
- Each protein has several **binding domains** and **modification sites**.
- Sites can be occupied or modified *in combination*.
- Most interactions depend on only one or two sites: SOCS binding does not care whether S727 is phosphorylated.
- Naming every combination is **combinatorial complexity**: a few interactions imply a huge number of species.

Combinatorial complexity in a realistic model of EGFR signaling

A model of the ErbB receptor family with its known binding partners:

- ErbB3:ErbB1 dimer: $> 3.8 \times 10^9$ states
- ErbB1:ErbB1 dimer: $> 5.5 \times 10^{10}$ states

Yet the model is built from a manageable number of **rules**, in the tens to hundreds.

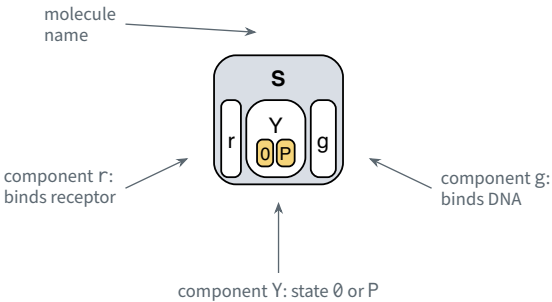


Creamer et al., *BMC Syst. Biol.* **6**, 107 (2012)

Part 2

Structured molecules in BNGL

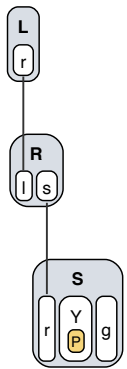
Molecules have components, and components can have states




```
begin molecule types
  S(r,Y~0~P,g)
end molecule types
```

- A **molecule** is a named object with a list of **components** in parentheses.
- Components represent **binding sites** or **modification sites**.
- ~ lists the allowed **states** of a component: `Y~0~P`.
- A component can have both a state and a bond.
- A molecule with no components, `mSOCS()`, is just a species name, as in Module 1.
- In the diagrams: molecules gray, components white, states yellow.

Complexes are molecules joined by bonds



The Module 1 species LRS_p written with structure:

```
L(r!1).R(l!1,s!2).S(r!2,Y~P,g)
```

- `.` joins molecules in the same complex.
- `!1` marks one end of a bond; the matching `!1` is the other end. Bond labels only need to be unique within a species.
- A component with no `!` is **free**: `g` above.
- A species is now a **graph**: molecules contain components, and bonds join components. BioNetGen recognizes when two strings describe the same graph.

Patterns select sets of species

A **pattern** lists only the parts of a complex you care about. **Don't write, don't care**: anything not mentioned can be in any state.

pattern	LR	LRS	LRS_p	S_p	GS_p	LR·SOCS	meaning
`S(Y~P)`	×	×	✓	✓	✓	×	any phosphorylated STAT
`S(r,Y~P)`	×	×	×	✓	✓	×	phosphorylated, not on a receptor
`S(r!+)`	×	✓	✓	×	×	×	STAT bound via `r`
`R(l!+)`	✓	✓	✓	×	×	✓	ligand-bound receptor
`R(l!+,s)`	✓	×	×	×	×	×	… with docking site free
`R(s!+)`	×	✓	✓	×	×	✓	docking site occupied

- `!+` bound to anything `!?` bound or not no `!` must be free
- Matching is done on the graph, so the pattern can match a molecule anywhere inside a complex.

Observables are patterns

Module 1: list every species by hand

```
Molecules pSTAT LRSp Sp GSp
```

Module 2: describe the property

```
Molecules pSTAT S(Y~P)
```

The pattern keeps working when we add new complexes containing pSTAT. No observable needs to be rewritten.

`Molecules` VS. `Species`

- `Molecules` counts every *match*. A STAT dimer with two pTyr contributes 2 to `S(Y~P)`.
- `Species` counts each matching *complex* once.

```
Species pSTATdimer S(d!1).S(d!1)
```

Rules transform patterns

```
Sbind:  R(l!+,s) + S(r,Y~0) <-> R(l!+,s!1).S(r!1,Y~0)  kSon, kSoff
```

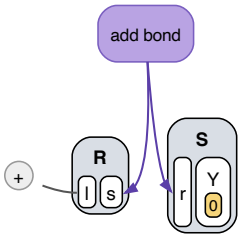
- **Reactant patterns** (left) say what must be present: a ligand-bound receptor with a free docking site, and an unphosphorylated STAT with a free `r` site.
- **Product patterns** (right) say what changes: a bond forms between `s` and `r`.
- Everything not mentioned is carried along unchanged.
- Rates follow mass action, with the *rule's* rate constant (times a statistical factor for identical molecules or sites).

What a rule can do

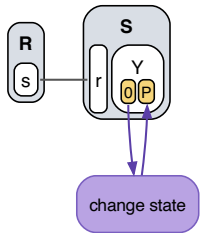
form a bond	`A(b) + B(a) -> A(b!1).B(a!1)`
break a bond	`A(b!1).B(a!1) -> A(b) + B(a)`
change a state	`S(Y~0) -> S(Y~P)`
make a molecule	`G(s!+) -> G(s!+) + mSOCS()`
delete one	`SOCS(r) -> 0`

Rule diagrams: what a rule changes, and where it applies

```
Sbind:  R(l!+,s) + S(r,Y~0) <-> R(l!+,s!1).S(r!1,Y~0)  kSon, kSoff
Sact:   R(s!1).S(r!1,Y~0) -> R(s!1).S(r!1,Y~P)         kact
```



Sbind



Sact

- **Purple** nodes are **operations**, with arrows to what they change: a new bond, or `Y` from `0` to `P`.
- Everything else is **context**: gray lines are bonds that must exist, `+` is a bond to any partner (`l!+`).
- A component drawn with no line must be free. One the rule does not mention (`g`) is not drawn.

Generated with `visualize({type=>"ruleviz_operation"})` (Sekar et al., 2017).

Part 3

The Module 1 model as rules

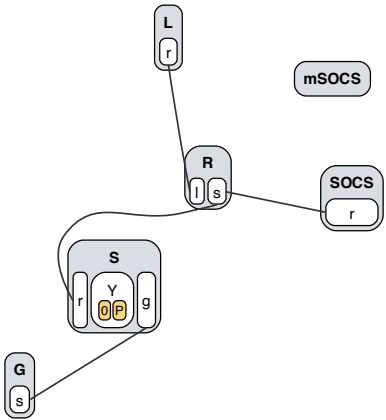
Molecule types, seed species, and observables

```
begin molecule types
  L(r)
  R(l,s)
  S(r,Y~0~P,g)
  G(s)
  mSOCS()
  SOCS(r)
end molecule types
begin seed species
  L(r)        L0
  R(l,s)      R0
  S(r,Y~0,g)  S0
  G(s)        G0
end seed species
begin observables
  Molecules pSTAT  S(Y~P)
  Molecules SOCS   SOCS()
end observables
```

- Six molecule types replace twelve species names.
- **Seed species** are the complexes present at $t = 0$. Every other species is generated from them by the rules.
- The receptor has one docking site `s` that STAT and SOCS compete for. This is the Module 1 assumption.

File: `models/stat_rbm1_rules.bngl`

The contact map: what can bind what



- Each molecule type appears once, with its components and states.
- Each line joins two components that some rule can bind.
- It shows what is *possible*, not *when*: STAT and SOCS both bind `s`, but the map does not say that only ligand-bound receptors recruit. That is the rules' context.

Generated with `visualize({type=>"contactmap"})`.

The rules

```
LRbind:     L(r) + R(l,s) <-> L(r!1).R(l!1,s)              kon, koff
Sbind:      R(l!+,s) + S(r,Y~0) <-> R(l!+,s!1).S(r!1,Y~0)  kSon, kSoff
Sact:       R(s!1).S(r!1,Y~0) -> R(s!1).S(r!1,Y~P)         kact
Srelease:   R(s!1).S(r!1,Y~P) -> R(s) + S(r,Y~P)           kdiss
Sdephos:    S(r,g,Y~P) -> S(r,g,Y~0)                       kdephos
Gbind:      S(r,Y~P,g) + G(s) <-> S(r,Y~P,g!1).G(s!1)      kGon, kGoff
Transcribe: G(s!+) -> G(s!+) + mSOCS()                     ktx
mRNAdeg:    mSOCS() -> 0                                   kmdeg
Translate:  mSOCS() -> mSOCS() + SOCS(r)                   ktl
SOCSdeg:    SOCS(r) -> 0                                   kSOCSdeg
SOCSbind:   R(l!+,s) + SOCS(r) <-> R(l!+,s!1).SOCS(r!1)    kSOCSon, kSOCSoff
```

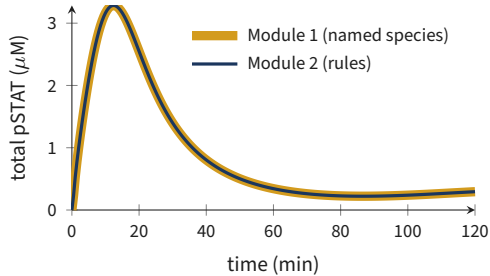
Context encodes the Module 1 assumptions. `R(l,s)` in `LRbind`: ligand binds and leaves only when the docking site is empty. `R(l!+,s)` in `Sbind` and `SOCSbind`: only ligand-bound receptors recruit.

Network generation: rules applied to seed species

1. Start with the seed species.
2. Apply every rule to every species (or pair of species) it matches.
3. Add any new species and reactions produced.
4. Repeat until nothing new appears.

`generate_network()` does this and writes the result to the `.net` file. ODE or SSA simulation then uses the generated network.

Result: the same 12 species and 15 reactions as Module 1.



The curves are identical.

Part 4

Relaxing the assumptions

1. Ligand can leave any receptor complex

Remove the context `s` from the ligand rule:

```
LRbind:  L(r) + R(l) <-> L(r!1).R(l!1)  kon, koff
```

Docking and activation still need bound ligand, but undocking does not, so each reversible rule is split in two:

```
Sbind:   R(l!+,s) + S(r,Y~0) -> R(l!+,s!1).S(r!1,Y~0)  kSon
Sunbind: R(s!1).S(r!1,Y~0) -> R(s) + S(r,Y~0)          kSoff
Sact:    R(l!+,s!1).S(r!1,Y~0) -> R(l!+,s!1).S(r!1,Y~P)  kact
```

(and likewise for SOCS)

One rule, many reactions

`LRbind` now generates 4 binding reactions: ligand binds R alone, R·S, R·S_p, or R·SOCS.

	species	reactions
before	12	15
after	**15**	**24**

2. SOCS binds its own site and blocks kinase activity

Give the receptor a separate SOCS site. SOCS no longer blocks docking; it blocks *activation*:

```
R(l,s,socs)

Sact:       R(l!+,socs,s!1).S(r!1,Y~0) ->
              R(l!+,socs,s!1).S(r!1,Y~P)  kact
SOCSbind:   R(l!+,socs) + SOCS(r) ->
              R(l!+,socs!1).SOCS(r!1)     kSOCSon
SOCSunbind: R(socs!1).SOCS(r!1) -> R(socs) + SOCS(r)  kSOCSoff
```

`socs` free in `Sact` means: activation only when SOCS is absent.

Closer to how SOCS proteins work: SOCS1 inhibits JAK directly, and SOCS3 inhibits JAK while bound to the receptor.

Now a receptor can carry STAT *and* SOCS at the same time.

	species	reactions
before	15	24
after	**19**	**39**

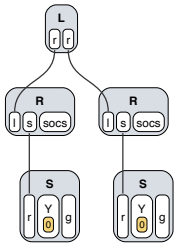
3. Cytokines bring two receptor chains together

In the simplest case, a bivalent ligand `L(r,r)` crosslinks two identical receptor chains, whose JAKs activate each other, so STAT is activated only in a dimer:

```
L(r,r)

LRbind:    L(r,r) + R(l) <-> L(r!1,r).R(l!1)      kon, koff
Crosslink: L(r!+,r) + R(l) <-> L(r!+,r!1).R(l!1)  kx, koff
Sact:      L(r!+,r!2).R(l!2,socs,s!1).S(r!1,Y~0) ->
             L(r!+,r!2).R(l!2,socs,s!1).S(r!1,Y~P)  kact
```

- BioNetGen accounts for the two identical ligand sites automatically (statistical factor of 2 in `LRbind`).
- Excess bivalent ligand occupies chains one at a time and blocks crosslinking, so we lower L_0 from 10 to 1 μM.



	species	reactions
before	19	39
after	**40**	**188**

`Crosslink` alone generates 36 reactions in each direction.

4. STATs act as dimers

Add a dimerization site. Free pSTAT dimerizes, only dimers bind the SOCS promoter, and only monomers are dephosphorylated:

```
S(r,Y~0~P,d,g)

Sdimer:  S(r,Y~P,d) + S(r,Y~P,d) <->
           S(r,Y~P,d!1).S(r,Y~P,d!1)          kdim, kundim
Gbind:   S(d!+,g) + G(s) <-> S(d!+,g!1).G(s!1)  kGon, kGoff
Sdephos: S(r,d,g,Y~P) -> S(r,d,g,Y~0)          kdephos
```

`S(d!+,g)`: any STAT that is in a dimer and not yet on DNA.

We restricted dimerization to free pSTAT. If receptor-bound STATs could dimerize too, dimers could bridge receptor complexes into chains of unlimited length: an **infinite** network.

	species	reactions
before	40	188
after	**43**	**198**

5. A second phosphorylation site doubles everything

STAT3 is also phosphorylated on Ser727. We assume this is independent of Tyr705; context could change that. One new component and *one* rule:

```
S(r,Y~0~P,S727~0~P,d,g)

Sser:  S(S727~0) <-> S(S727~P)  kser, kserdephos
```

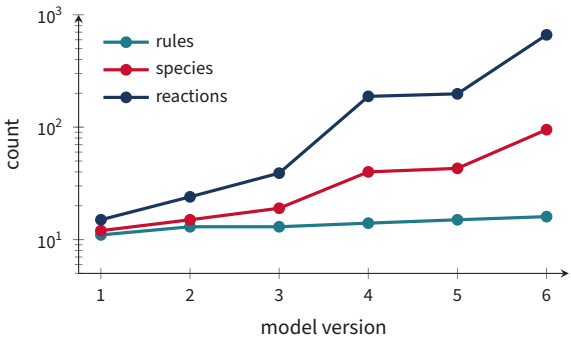
The rule mentions only `S727`, so it applies to STAT in *every* complex: free, receptor-bound, in a dimer, on DNA.

Every species containing one STAT now comes in 2 versions, and every species containing two STATs in 3 or 4.

	species	reactions
before	43	198
after	**95**	**663**

`Sser` alone generates 59 forward and 59 reverse reactions.

Rules grow linearly; the network grows combinatorially

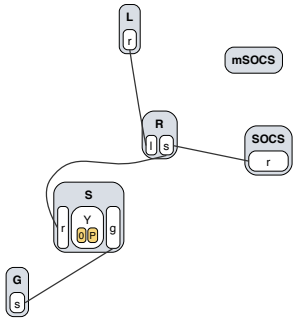


1 Module 1 as rules
2 ligand leaves any complex
3 separate SOCS site
4 receptor dimers
5 STAT dimers
6 Ser727 site

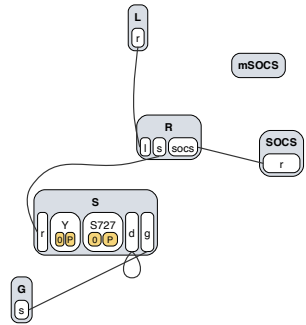
From 11 to 16 rules, the network grows from 15 to 663 reactions.

Writing these 663 reactions, and the ODEs for 95 species, by hand would be slow and error-prone. **Rules let us write down the mechanism and let the software do the bookkeeping.**

The contact map grows like the rules



version 1: 8 components, 4 possible bonds



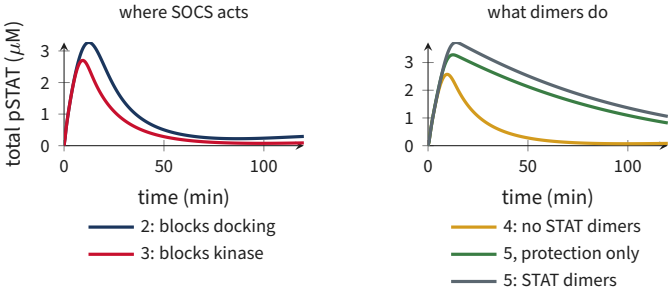
version 6: 11 components, 5 possible bonds

The network grew from 15 to 663 reactions.

The contact map, like the rules, stays small enough to read at a glance.

Mechanistic details change the predictions

Compare models that differ in *one* mechanism.



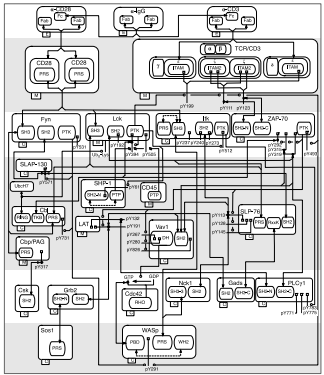
- Blocking the kinase lowers the peak and speeds the shutoff.
- Version 5 changes two things. Protection from the phosphatase accounts for most of the longer response (half-decay 22 → 70 min, 75 min with both).
- Experiments decide which mechanism is right. Rules make the alternatives easy to build and compare.

Part 5

Scaling up

Rules capture knowledge at large scale

T cell receptor model



20 molecule types
129 rules

Number of species is too large for network generation!

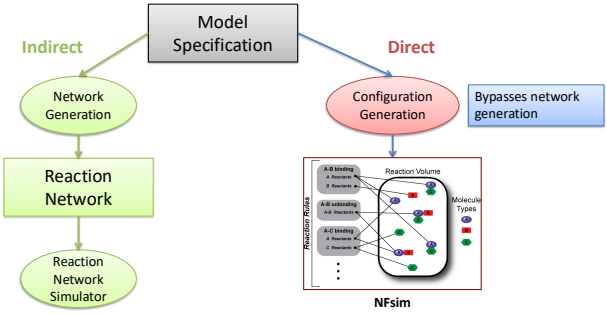
A rule-based model of T cell receptor signaling: 20 molecule types and 129 rules.

The number of possible species is too large to enumerate, so the network cannot be generated at all.

Rule-based models also serve as precise, extensible, visualizable representations of what is known about a pathway.

Chylek et al., *PLOS One* (2014)

Two ways to simulate a rule-based model

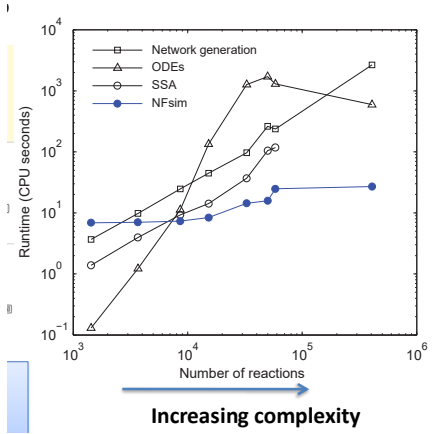


Indirect: generate the network, then integrate ODEs or run SSA on it. Fast when the network is small. This is what we have done so far.

Direct (network-free): track individual molecules and apply rules to them as the simulation runs. Only the complexes that actually form are ever represented.

`simulate({method=>"nf",...})` runs NFsim.

Network-free simulation scales with the rules, not the network



- The cost of network generation and ODE integration grows with the number of reactions.
- NFsim's cost per event depends on the number of rules and molecules, so it handles models whose networks are huge or infinite.
- Trade-off: NFsim is stochastic and works with molecule counts, so results are noisy and concentrations must be converted to numbers.

Sneddon, Faeder, and Emonet, *Nat. Methods* **8**, 177 (2011)

Try it yourself

Using `models/stat_rbm*.bngl`:

1. In `stat_rbm1_rules.bngl`, change `R(l,s)` to `R(l)` in `LRbind`. Generate the network. Which new species appear, and why?
2. Write an observable for receptors that carry both STAT and SOCS in `stat_rbm3_socs_site.bngl`.
3. In `stat_rbm4_receptor_dimer.bngl`, vary `L0` from 0.01 to 100. Plot peak pSTAT against `L0`. Explain the shape.
4. Remove the `r` context from `Sdimer` in `stat_rbm5_stat_dimer.bngl` so receptor-bound pSTAT can dimerize. What happens when you generate the network? Try `generate_network({max_agg=>6})`.

No local setup yet? Paste any of these models into BNG Playground (`ruleworld.github.io/bngplayground`) and run it in your browser.

Summary

- Multi-site proteins create **combinatorial complexity**: a few interactions imply many species and reactions.
- BNGL represents molecules as objects with **components** (sites), **states**, and **bonds**; species are graphs.
- **Patterns** specify only what matters (don't write, don't care) and define observables that stay correct as the model grows.
- **Rules** transform patterns. One rule generates every reaction that fits its pattern; context restricts where it applies.
- BioNetGen **generates the network** from rules for ODE/SSA simulation; **NFsim** simulates models whose networks are too large to generate.